Tema 3: Registros, Vectores y Matrices

Fundamentos de Programación

Francisco J. Rodríguez Díaz
(fjrodriguez@decsai.ugr.es)
Departamento de Ciencias de la Computación e I.A.

Universidad de Granada

Tabla de Contenidos I

1. Registros
2. Vectores. Algoritmos de búsqueda y ordenación sobre vectores
3. Matrices

Contenido del tema

1. Registros
2. Vectores. Algoritmos de búsqueda y ordenación sobre vectores
3. Matrices

Registros

Registros

- Hasta ahora hemos visto cómo usar variables de los tipos básicos del lenguaje: int, float, double, string, ...
- Eso no es cómodo para trabajar.

Manejando puntos

Vamos a hacer un programa que usa varios puntos.

```
1 double punto1_x, punto1_y;
2 double punto2_x, punto2_y;
3
4 punto1_x = ...
5 punto1_y = ...
6 punto2_x = ...
7 punto2_y = ...
8 ...
```

Registros

Registros

- Hasta ahora hemos visto cómo usar variables de los tipos básicos del lenguaje: int, float, double, string, ...
- Eso no es cómodo para trabajar.

Manejando puntos

Vamos a hacer un programa que usa varios puntos.

```
1 double punto1_x, punto1_y;
2 double punto2_x, punto2_y;
3
4 punto1_x = ...
5 punto1_y = ...
6 punto2_x = ...
7 punto2_y = ...
8 ...
```

Trabajar los datos así sería muy tedioso.

Registros en C/C++

Registros

- Se pueden definir nuevos tipos de datos **compuestos** a partir de atributos.
- Cada atributo tiene un tipo y un nombre.
- Una vez definido el tipo se pueden declarar variables.

Formato

```
struct <TipoDato> {
    <tipo1> <nombre1>;
    <tipo2> <nombre2>;
    ...
    <tipoN> <nombreN>;
};
```

Uso de registros

Definir una variable

Se puede definir una variable del nuevo tipo con:

`<TipoDato> <nombreVariable>;`

Ejemplo de registro

```
1  struct Punto {
2     double x;
3     double y;
4  };
5
6  int main() {
7     Punto p1, p2;
8     ...
9  }
```

Ejemplo completo

Uso del registro

Se puede usar los campos de la variable con:

`<nombreVariable>.<nombreAtributo>;`

Ejemplo de registro

```
1   Punto p1, p2, p3;
2
3   cin >> p1.x;
4   cin >> p1.y;
5   cin >> p2.x;
6   cin >> p2.y;
7
8   //asignación
9   p3 = p2;
10
11  //uso en operaciones dependiendo del tipo (aritméticas)
12  p3.x = p1.x + p2.x;
13  p3.y = p1.y + p2.y;
14
15  cout <<"p3: " <<p3.x <<", " <<p3.y <<endl;
```

Registro con campos heterogéneos

Datos empleado

```
1    struct Empleado {
2     string nombre;
3     string apellidos;
4     string NIF;
5     char categoria;
6     double salario_bruto;
7  };
```

Inicialización en la declaración

```
1    Empleado un_empleado = {"", "", "", 'A', 0.0};
```

Contenido del tema

1. Registros
2. Vectores. Algoritmos de búsqueda y ordenación sobre vectores
3. Matrices

Vectores

Motivación

Ha menudo es conveniente trabajar con un número variable de datos.

Ejemplo

Leer tres calificaciones, y decir cuántas son mejores que la media.

Vectores

Motivación

Ha menudo es conveniente trabajar con un número variable de datos.

Ejemplo

Leer tres calificaciones, y decir cuántas son mejores que la media.

Problema

- Se deben leer todas las calificaciones para calcular la media.
- Luego se compara calificación por calificación.

Primera versión

```
5  int main(void)
6  {
7    int superan_media;
8    double nota1, nota2, nota3, media;
9
10   do {
11     cout <<"Introduce nota1: ";
12     cin >>nota1;
13   } while (nota1 < 0 || nota1 > 10);
14
15   do {
16     cout <<"Introduce nota2: ";
17     cin >>nota2;
18   } while (nota2 < 0 || nota2 > 10);
19
20   do {
21     cout <<"Introduce nota3: ";
22     cin >>nota3;
23   } while (nota3 < 0 || nota3 > 10);
```

Primera versión (parte dos)

```
25    media = (nota1 + nota2 + nota3) / 3.0;
26    superan_media = 0;
27
28    if (nota1 > media) {
29      superan_media++;
30    }
31
32    if (nota2 > media) {
33      superan_media++;
34    }
35
36    if (nota3 > media) {
37      superan_media++;
38    }
39
40    cout <<"Superan la media: " <<superan_media <<endl;
41    return 0;
42  }
```

Primera versión (parte dos)

```
25    media = (nota1 + nota2 + nota3) / 3.0;
26    superan_media = 0;
27
28    if (nota1 > media) {
29      superan_media++;
30    }
31
32    if (nota2 > media) {
33      superan_media++;
34    }
35
36    if (nota3 > media) {
37      superan_media++;
38    }
39
40    cout <<"Superan la media: " <<superan_media <<endl;
41    return 0;
42  }
```

¿Y si fuesen 80 notas?

Vectores

Necesidad

Sería necesario poder gestionar de forma sencilla varios datos.

C++ permite el tipo **Vector** (o *array*) que permite gestionar fácilmente varios valores.

Vector

Es un **tipo de dato** compuesto de un **número fijo** de componentes del mismo tipo.

Acceso

Se puede acceder por un índice, valor entero (el primero es 0).

Antes y después

Antes

```
8   double nota1, nota2, nota3, media;
9
10  do {
11    cout <<"Introduce nota1: ";
12    cin >>nota1;
13  } while (nota1 < 0 || nota1 > 10);
14
15  do {
16    cout <<"Introduce nota2: ";
17    cin >>nota2;
18  } while (nota2 < 0 || nota2 > 10);
19
20  do {
21    cout <<"Introduce nota3: ";
22    cin >>nota3;
23  } while (nota3 < 0 || nota3 > 10);
```

Antes y después

Después

```
8   double notas[3];
9   double media;
10
11  for (int i = 0; i < 3; i++) {
12    do {
13      cout <<"Introduce nota" <<(i+1) <<": ";
14      cin >>notas[i];
15    } while (notas[i] < 0 || notas[i] > 10);
16
17  }
```

Terminando el código

Después

```
19    media = 0;
20
21    for (int i = 0; i < 3; i++) {
22      media += notas[i];
23    }
24    media /= 3.0;
25    superan_media = 0;
26
27    for (int i = 0; i < 3; i++) {
28      if (notas[i] > media) {
29        superan_media++;
30      }
31    }
32
33    cout <<"Superan la media: " <<superan_media <<endl;
34    return 0;
35  }
```

Declaración de un vector

Formato

```
<tipo> <identificador> [<numero_componentes>];
```

- **tipo** es el tipo de cada componente (siempre es el mismo).
- **numero_componentes** número de componentes o tamaño del vector. Debe conocerse en tiempo de compilación, y no se puede cambiar (literal o constante).

Declaración de un vector

Formato

```
<tipo> <identificador> [<numero_componentes>];
```

- **tipo** es el tipo de cada componente (siempre es el mismo).
- **numero_componentes** número de componentes o tamaño del vector. Debe conocerse en tiempo de compilación, y no se puede cambiar (literal o constante).

Cuidado

Algunos compiladores permiten que el número sea una variable, pero no es parte del estándar, no lo aceptamos como válido.

Tamaño del vector

```
const int MAX_ALUMNOS = 30;
int valores[MAX_ALUMNOS];
```

Mayor valor

- Permite trabajar con más datos.
- Ocupa más memoria (aunque no se use).

Menor valor

- Ocupa menos memoria.
- Limita mucho más el número de elementos.

Uso de constantes

Uso de constantes

Es conveniente el uso de constantes en lugar de literales para el tamaño

Ejemplo

```
1 // Así si
2 const int MAX_ALUMNOS = 3;
3 double notas[MAX_ALUMNOS];
4 int variable = 3;
5 double otrasnotas[variable]; // Error de compilación
```

Espacio en memoria

Variable

```
1 int notas[3];
```

Posicion en memoria

Si cada entero son 4 bytes, la estructura en memoria es:

Espacio en memoria

Variable

```
1 int notas[3];
```

Posicion en memoria

Si cada entero son 4 bytes, la estructura en memoria es:

0 1 2 3	4 5 6 7	8 9 10 11
notas[0]	notas[1]	notas[2]

Estructura

- Primer dato en los primeros 4 bytes (0-3).
- Segundo dato en los siguientes 4 bytes (4-7).
- Tercer dato en los siguientes 4 bytes (8-12).

Espacio en memoria

Variable

```
1 int notas[3];
```

Posicion en memoria

Si cada entero son 4 bytes, la estructura en memoria es:

0 1 2 3	4 5 6 7	8 9 10 11
notas[0]	notas[1]	notas[2]

Estructura

- Primer dato en los primeros 4 bytes (0-3).
- Segundo dato en los siguientes 4 bytes (4-7).
- Tercer dato en los siguientes 4 bytes (8-12).

Total

3*sizeof(int)=12 bytes.

Espacio en memoria

Variable

```
1 char DNI[9];
```

Posicion en memoria

Si cada caracter es un byte, la estructura en memoria es:

Espacio en memoria

Variable

```
1 char DNI[9];
```

Posicion en memoria

Si cada caracter es un byte, la estructura en memoria es:

0	1	2	3	4	5	6	7	8
DNI[0]	DNI[1]	DNI[2]	DNI[3]	DNI[4]	DNI[5]	DNI[6]	DNI[7]	DNI[8]

Estructura

- Cada elemento es un byte.

Espacio en memoria

Variable

```
1 char DNI[9];
```

Posicion en memoria

Si cada caracter es un byte, la estructura en memoria es:

0	1	2	3	4	5	6	7	8
DNI[0]	DNI[1]	DNI[2]	DNI[3]	DNI[4]	DNI[5]	DNI[6]	DNI[7]	DNI[8]

Estructura

- Cada elemento es un byte.

Total

9*sizeof(char)=9 bytes.

Accediendo a los datos

Definición

Si se ha definido como:

```
<tipo> <identificador>[<numero_componentes>];
```

Acceso

Se accede mediante:

```
<identificador> [<indice>];
```

- **identificador** es el nombre de la variable.
- **indice** es un valor numérico entre 0 y **numero componentes-1**.

Uso de cada elemento

Uso

- Cada componente se puede usar como una variable más.
- Su tipo es el indicado al declarar el vector.

Ejemplo de uso

```
1  double suma_tres = notas[0]+notas[1];
2  double nota_min;
3
4  if (notas[0] < notas[1]) {
5     nota_min = notas[0];
6  }
7  else {
8     nota_min = notas[1];
9  }
```

Asignación del vector

Cuidado

- No se puede asignar valores a todo el vector.
- Es necesario asignar elemento a elemento.

Mal ejemplo

```
7   const int MAX_VALORES=3;
8   int valores[MAX_VALORES];
9
10  valores = {1,2,3}; // No compila
```

Asignación del vector

Opción 1

La primera opción es iniciar elemento a elemento.

```
12   valores[0] = 1;
13   valores[1] = 2;
14   valores[2] = 3;
```

Alternativa

La otra opción es asignarlo al definir la variable.

```
7   const int MAX_VALORES=3;
8   int valores[MAX_VALORES]= {1,2,3};
```

Cuidado al asignar

Cuidado

Es conveniente que los valores se repitan tanto como valores.

Cuidado

```
1 const MAX_ALUMNOS = 3;
2 // Deja la 3ª sin asignar
3 int notas[MAX_ALUMNOS] = {7, 6};
4 // Deja la 2ª y 3ª sin asignar
5 int notas[MAX_ALUMNOS] = {7};
```

Ejemplos de uso

Ejemplo de acceso

Se puede acceder a partir de una variable

```
int pos = 2;
notas[pos] = notas[pos]+0.5;
```

Ejemplo de uso de vector

Eso permite recorrerlos en un bucle.

```
// Sumar medio punto a todas las calificaciones
for (int i = 0; i < MAX_ALUMNOS; i++) {
    notas[i] += 0.5;
}
```

Lectura y escritura

Componente a componente

Se lee o escribe elemento a elemento.

Lectura

```
1 for (int i = 0; i < MAX_ALUMNOS; i++) {
2     cout <<"Introduce la nota del alumno " <<(i+1) <<": ";
3     cin >>notas[i];
4 }
```

Lectura con validación

```
1 for (int i = 0; i < MAX_ALUMNOS; i++) {
2     do {
3        cout <<"Introduce la nota del alumno " <<(i+1) <<": ";
4        cin >>notas[i];
5     } while(notas[i] < 0 || notas[i] > 10);
6 }
```

Cuidado con los índices

Acceso

C++ no verifica que el índice sea correcto.

Valores

```
1 int notas[3];
```

Memoria

notas	??	7	6	9	??
i	-1	0	1	2	3

- notas[-1] vale basura.
- notas[0] vale 7.
- notas[2] vale 9.
- notas[3] vale basura.

El tipo string como secuencia de caracteres

Acceso con operador corchete

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string cadena;
8      cadena = "FundamentAs de Programación";
9      cout << cadena[0]; // Imprime F
10     cadena[9] = 'o'; // Fundamentos de Programación
11     cadena[30] = 'o'; // Error: comportamiento indeterminado
12 }
```

Otras operaciones

- **cadena.size()** devuelve el tamaño de la cadena
- **cadena.push_back(caracter)** añade un caracter al final de la cadena
- **+** operador de concatenación

El tipo string como secuencia de caracteres

Ejemplo

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6      string cadena;
7      cadena = "Fundamento";
8
9      cout << cadena.size(); // 10
10     cadena.push_back('s');
11     cout << cadena; // Fundamentos
12     cout << cadena.size(); // 11
13     cadena = cadena + " de Programación"; // cadena += " de Programación"
14     cout << cadena; // Fundamentos de Programación
15     cadena[50] = 'o'; // Error. Usar push_back para añadir caracteres
16 }
```

Gestión de las componentes utilizadas

Huecos en cualquier posición con marcador especial N

X	X	N	X	X	N	X	···	X

Ejemplo

```
1  int main() {
2      const double NULO = -1;
3      const int MAX_ALUMNOS = 100;
4      double notas[MAX_ALUMNO];
5      double media = 0, suma = 0;
6      int num_alumnos = 0;
7
8      for (int i = 0; i < MAX_ALUMNOS; i++) {
9          if (notas[i] != NULO) {
10             suma += notas[i];
11             num_alumnos++;
12         }
13     }
14     media = suma / num_alumnos;
15 }
```

Gestión de las componentes utilizadas

Huecos a la derecha sin terminador

Se usa una variable entera para indicar el número de componentes usadas (Ejemplo: util_vector = 7). Esquema muy habitual a lo largo de la asignatura.

X	X	X	X	X	X	X	?	?	···	?

Ejemplo I

```
1     util_vector = 0;
2     cout << "Introduce nota del alumno (-1 para terminar): " << endl;
3     cin >> nota;
4
5     while (nota != -1) {
6         notas[util_vector] = nota;
7         util_vector++;
8         cout << "Introduce nota del alumno (-1 para terminar): " << endl;
9         cin >> nota;
10    }
```

Gestión de las componentes utilizadas

Ejemplo II

```
1    for (int i = 0; i < util_vector; i++) {
2        suma += notas[i];
3    }
4    media = suma / util_vector;
```

Vectores de registros

Datos empleado

```
1    struct Empleado {
2     string nombre;
3     string apellidos;
4     string NIF;
5     char categoria;
6     double salario_bruto;
7  };
```

Vectores de registros

Vector de empleados

```
Empleado lista_empleados[MAX_EMPLEADOS];

//lectura de los datos

cin >> lista_empleados[0].nombre;

lista_empleados[0].salario_bruto = 25000;


//mostrar por pantalla

for (int i = 0; i < num_empleados; i++) {
    cout << lista_empleados[i].nombre << endl;
    cout << lista_empleados[i].apellidos << endl;
    //....
}
```

Algoritmos de búsqueda y ordenación sobre vectores

Búsqueda

Determinar si un elemento está almacenado en el vector y acceder a dicho elemento.

- Secuencial
- Binaria

Ordenación

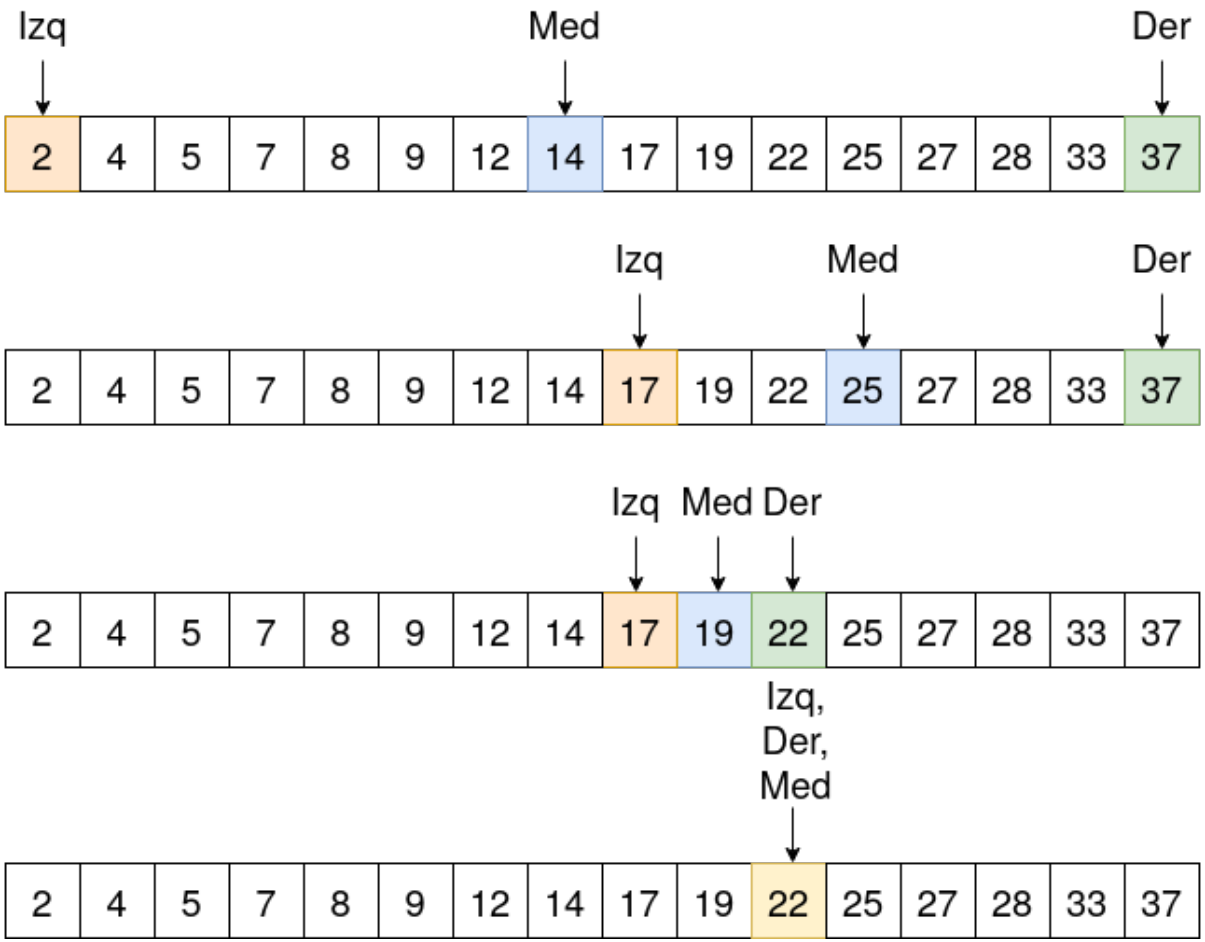
Ordenar los elementos almacenados en el vector de acuerdo a un determinado criterio.

- Selección
- Burbuja
- Inserción
- Hay más ...

Búsqueda Secuencial

```
1  int valores[TAM_MAX];
2  int usados, buscado, posicion_buscado;
3  bool encontrado = false;
4
5  for (int i = 0; i < usados && !encontrado; i++) {
6      if (valores[i] == buscado) {
7          encontrado = true;
8          posicion_buscado = i;
9      }
10 }
```

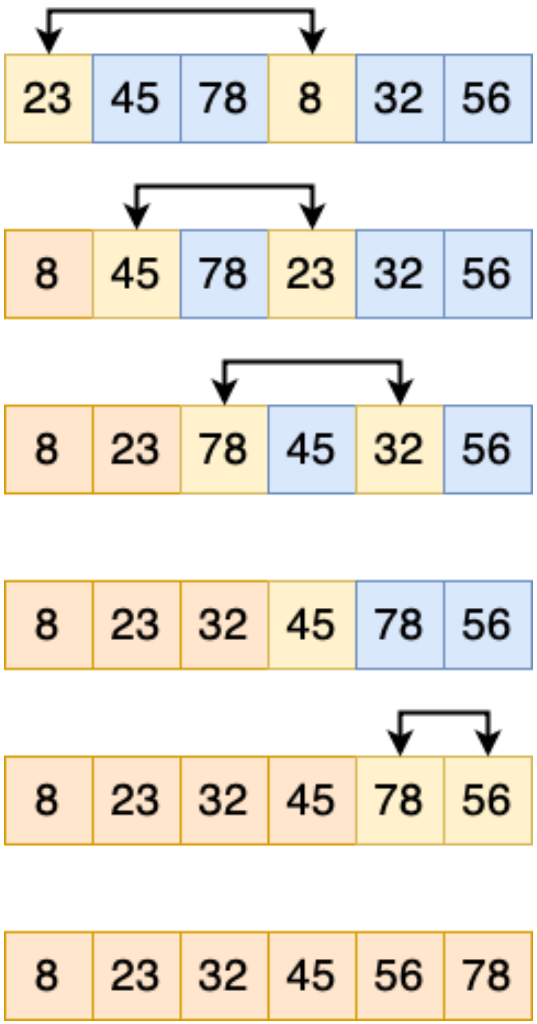
Búsqueda Binaria



Búsqueda Binaria

```
while (izquierda <= derecha && !encontrado) {
    centro = (izquierda + derecha) / 2;
    if (valores[centro] < buscado) {
        izquierda = centro + 1;
    }
    else if (valores[centro] > buscado) {
        derecha = centro - 1;
    }
    else {
        encontrado = true;
        posicion_buscado = centro;
    }
}
```

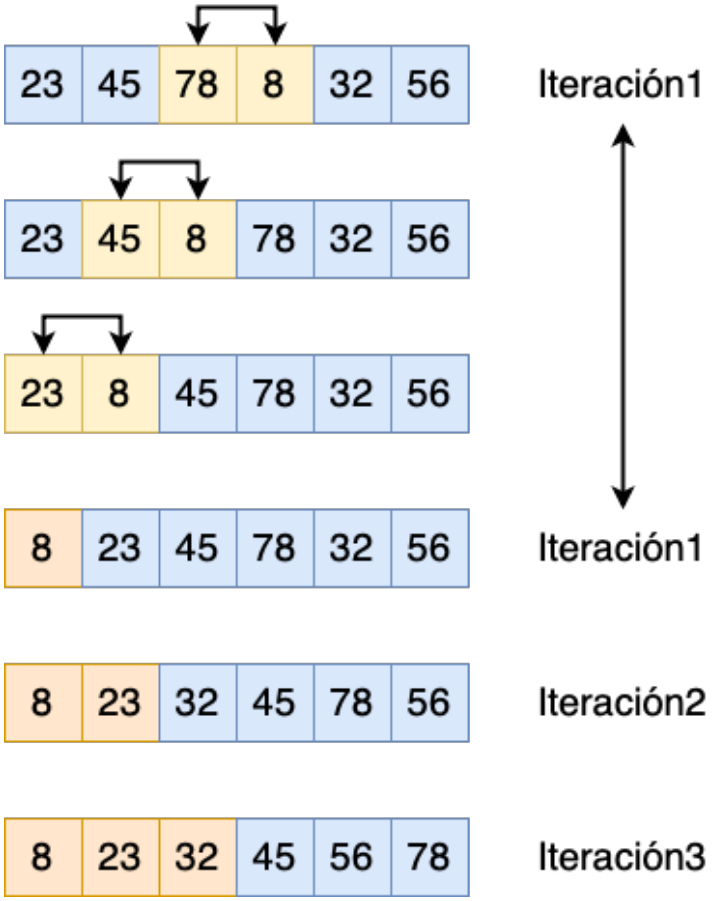
Ordenación por Selección



Ordenación por Selección

```
1  for (int i = 0; i < n; i++) {
2      minimo = i;
3      for (int j = i + 1; j < n; j++) {
4          if (v[j] < v[minimo]) {
5              minimo = j;
6          }
7      }
8      aux = v[i];
9      v[i] = v[minimo];
10     v[minimo] = aux;
11 }
```

Ordenación por Burbuja

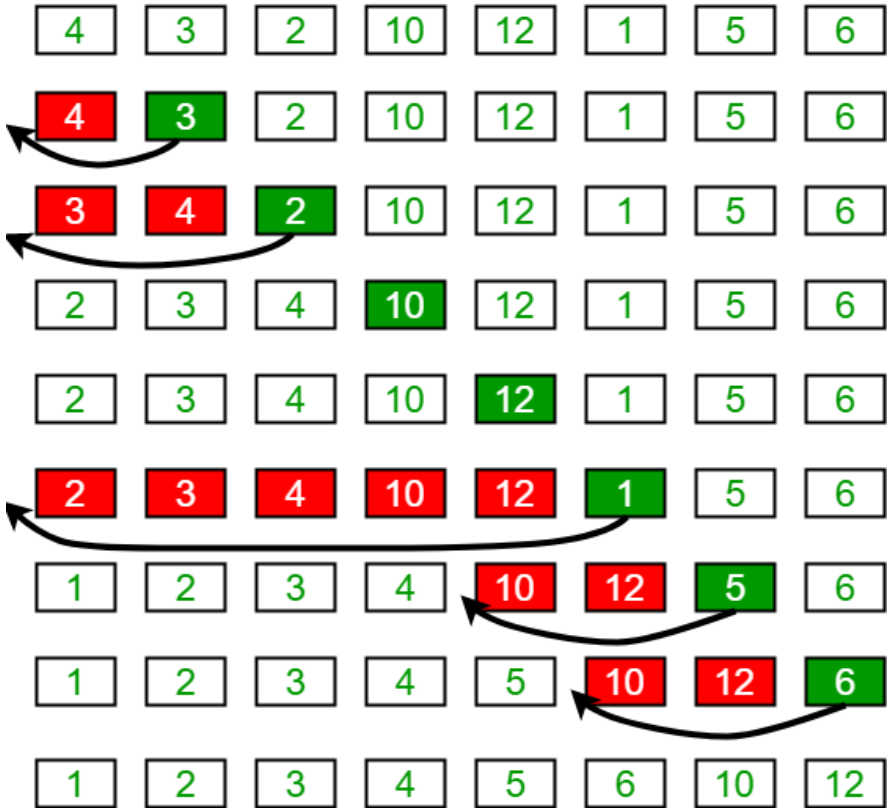


Ordenación por Burbuja

```
i = 0;

while (hay_cambios) {
    hay_cambios = false;
    for (int j = n - 1; j > i; j--) {
        if (v[j-1] > v[j]) {
            aux = v[j];
            v[j] = v[j-1];
            v[j-1] = aux;
            hay_cambios = true;
        }
    }
    i++;
}
```

Ordenación por Inserción



Ordenación por Inserción

```
1  for (int i=1; i<n; i++) {
2      clave = v[i];
3      j = i-1;
4      while (j >= 0 && v[j] > clave) {
5          v[j+1] = v[j];
6          j--;
7      }
8      v[j+1] = clave;
9  }
```

Contenido del tema

1. Registros
2. Vectores. Algoritmos de búsqueda y ordenación sobre vectores
3. Matrices

Declaración de una matriz

Formato

```
<tipo> <identificador> [<numero_filas>][<numero_columnas>];
```

Ejemplos

```
1 int imagen[MAX_ALTURA][MAX_ANCHURA];
2 double notas[NUM_ALUMNOS][NUM_NOTAS];
3 char crucigrama[MAX_FIL][MAX_COL];
```

Acceso y Asignación I

Lectura

```
<identificador> [<índice_fila>][índice_columna];
```

Asignación

```
<identificador> [<índice_fila>][índice_columna]=
    <expresión>;
```

Acceso y Asignación II

Ejemplo

```
1  int main() {
2    const int TAMANIO_FIL = 2;
3    const int TAMANIO_COL = 3;
4    double matriz[TAMANIO_FIL][TAMANIO_COL];
5
6    matriz[0][1] = 4.5 // correcto
7    matriz[2][0] = 7.2 // Error: Fila 2 no existe
8    matriz[0][3] = 7.2 // Error: Columna 3 no existe
9    matriz[1][2] = matriz[0][1] // correcto, toma el valor 4.5
10
11 }
```

Inicialización

Ejemplo

```
1  int matriz[2][3] = {{1,2,3},{4,5,6}}; // 1 2 3
2                                        // 4 5 6
3
4  int otra_matriz[2][3] = {{1},{3,4,5}}; // 1 0 0
5                                          // 3 4 5
6
7  int mas_matriz[3][3] = {1,2,3,4,5}; // 1 2 3
8                                      // 4 5 0
9                                      // 0 0 0
10                                                                    // 4
11 }
```

Representación en memoria

Representación

Depende del lenguaje de programación. En C++ se hace por filas.

Ejemplo

```
1  int m[2][3] = {{1},{3,4,5}}; // 1 0 0
2                               // 3 4 5
```

?	1	0	0	3	4	5	?	?

Más de dos dimensiones

Representación

Tantas dimensiones como queramos, simplemente añadiendo más corchetes

Ejemplo

```
1  const int MAX_FILAS = 3;
2  const int MAX_COLUMNAS = 5;
3  const int MAX_PROFUNDIDAD = 2;
4
5  int tresD[MAX_FILAS][MAX_COLUMNAS][MAX_PROFUNDIDAD];
6
7  tresD[0][3][1] = 7;
```

Trabajando con matrices I

Imágenes

Una imagen en blanco y negro es una matriz de píxeles, cada posición representa el nivel de gris de ese pixel.

-1	-1	-7	1	9	1
-2	-2	-1	8	-1	1
-6	-1	-3	1	3	1
-1	-2	-5	4	4	4
1	5	5	5	5	1

Trabajando con matrices II

Problema

Tenemos valores incorrectos (negativos). Reemplazar por la media de sus 8 vecinos, excluyendo los valores negativos. Por simplicidad no vamos a considerar los bordes.

Ejemplo

```
1  for (fil = 1; fil < util_fil - 1; fil++) {
2    for (col = 1; col < util_col - 1; col++) {
3      if (imagen[fil][col] < 0) {
4        suma = 0;
5        valores_usados = 0;
6
7        for (int i = fil - 1; i <= fil + 1; i++) {
8          for (int j = col - 1; j <= col + 1; j++) {
9            if (imagen[i][j] >= 0) {
10             suma = suma + imagen[i][j];
11             valores_usados++;
12           }
13         }
14       }
```

Trabajando con matrices III

Ejemplo

```
      if (valores_usados > 0 ) {
        imagen[fil][col] = suma / valores_usados;
      }
      else {
        imagen[fil][col] = 0;
      }

    }
  }
}
```